STM32 ile EBYTE LoRa Örneği (E32-900T20D + CP2102 TTL)

Lora nedir?

LoRa (Long Range) modülleri, düşük güç tüketimiyle uzun mesafelerde kablosuz veri iletimi sağlayan, genellikle IoT (Nesnelerin İnterneti) uygulamalarında kullanılan RF (radyo frekansı) iletişim cihazlarıdır. LoRa, fiziksel katmanda çalışarak verileri chirp spread spectrum (CSS) tekniğiyle iletir ve bu sayede birkaç kilometreye kadar menzil sunabilir. Düşük bant genişliği kullanarak enerji tasarrufu sağlarken, düşük veri hızlarında güvenilir bağlantı sağlar. LoRa modülleri genellikle mikrodenetleyicilerle UART, SPI veya I2C üzerinden haberleşir ve sensör verilerinin merkezi bir LoRaWAN ağına ya da doğrudan diğer modüllere iletilmesinde kullanılır.

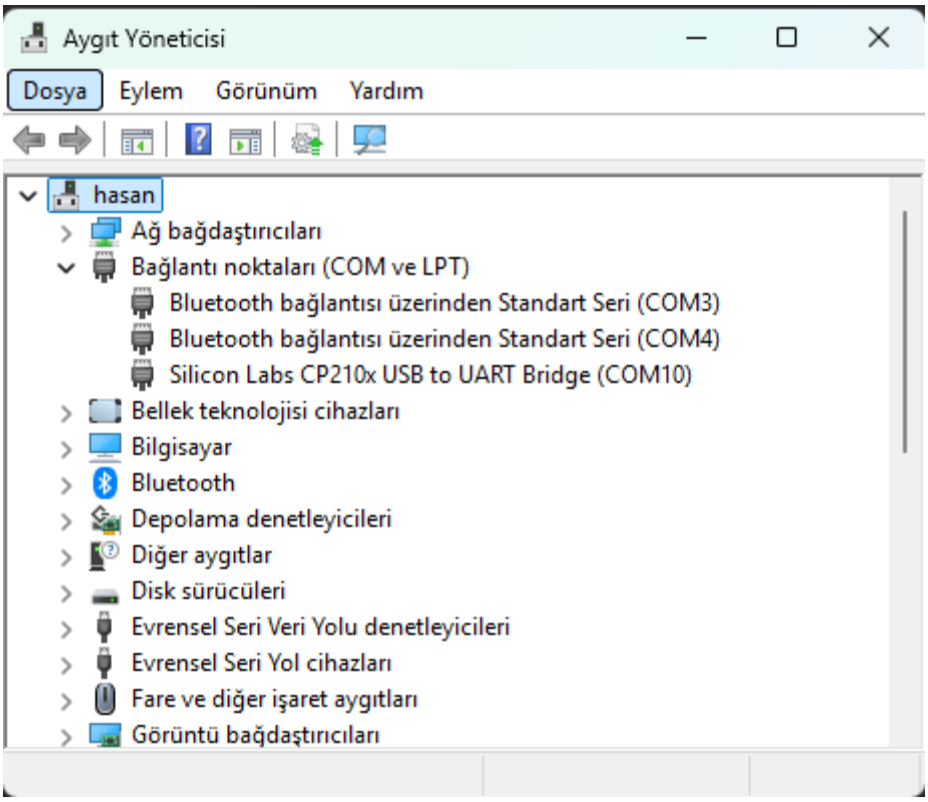
bu uygulama için ben 2 adet E32-900T20D kullandım. bir adet STM32F103C8T6 ile bilgisayarımı haberleştirmek üzere bir adet de CP2102 USB-TTL dönüştürücü kullandım. Burada ilk kez lora kullanılacak ise hangi adımların gerçekleştirilmesi gerektiğinden bahsedelim.

sizler kendi uygulamanızda isterseniz farklı bir usb-ttl converter kullanabilirsiniz. kullandığınız usb-ttl dönüştürücüye göre bilgisayarınıza uygun driverları kurmanız gerekmektedir. ben CP2102 kullandığım için aşağıdaki linkten driveri kurdum.

https://www.silabs.com/software-and-tools/usb-to-uart-bridge-vcp-drivers?tab=downloads

kurduktan sonra

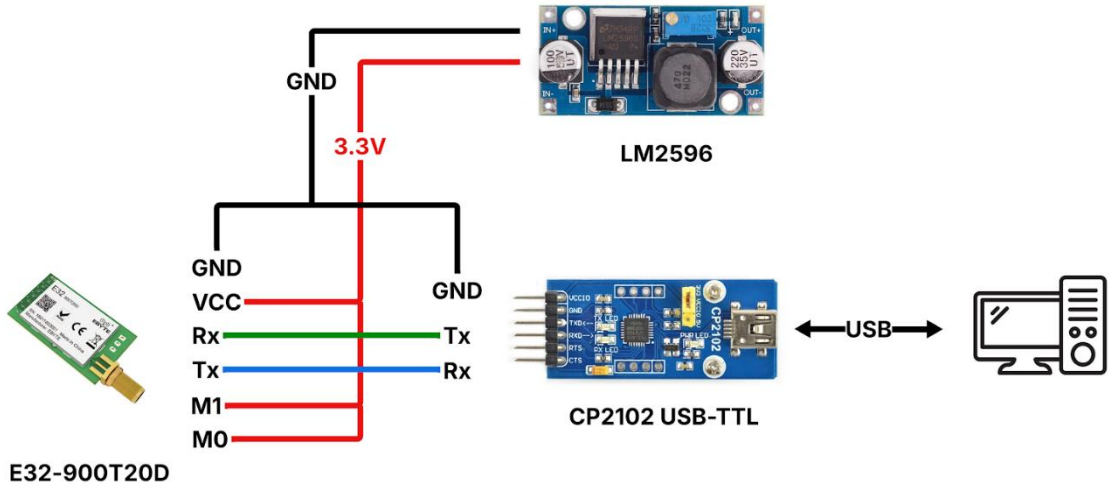
Bu Bilgisayar > Özellikler > Aygıt Yöneticisi kısmına gelip com portunuzu görebilmeniz gerekmektedir. Eğer ki göremiyorsanız ya usb-ttl takılı değildir ya da driverlar eksik ya da yanlış kurulmuştur.



USB-TTL dönüştürücü cihazını bilgisayarın herhangi bir USB portuna taktıktan sonra burada doğrudan COM port gözükecektir. USB-TTL converteri bilgisayara taktığımıza göre artık konfigüre etmek için LoRa modülümüzü bağlamamız gerekmektedir.

LoRa modülü çok fazla güç çektiği için USB-TTL converterin zarar görme olasılığından ötürü 3.3V'a regüle edilmiş harici bir güç kaynağından beslemek fayda olacaktır. 4V – 40V arası giriş gerilimiyle beslenebilen LM2596'nın üzerindeki pot ile regülatörün 3.3 volt çıkış vermesini sağlayabilecek şekilde değiştirilmesi gerekmektedir. **5 volt ve üzeri bir değer LoRa modülüne zarar verecektir.**

konfigürasyon için sistem şeması aşağıdaki gibidir. M0 ve M1 pinlerinin VCC ile ortaklanması gerekmektedir. bu durumda modülümüz konfigüre edilebilir. aksi halde modül konfigürasyon arayüzüne bağlanamaz. bununla beraber USB-TTL dönüştürücü ile LoRa modülünün Tx Rx kanallarının çapraz bağlanması gerekmektedir.



burada üretici EBYTE firma'nın sitesinden indirmeniz gereken dökümanlar ulaşılabilir. elinizdeki modüle göre indirmeniz gerekmektedir.

https://www.ebyte.com/datadown/canshuruanjian.html bu siteden gerekli olan modülü indirebilirsiniz. buradan RF_SettingV4.7 indirdim. sizler farklı modüllere göre farklı sürümlere ihtiyaç duyabilirsiniz.

https://www.ebyte.com/product-selecte.html bu siteden modül ile ilgili daha detaylı dokümanlara erişebilirsiniz. örneğin modülün 3D .step uzantılı görüntüsü teknik detaylarına vs buradan erişilebilir.

Models butonuna bastığınız takdirde programın desteklediği modüller gözükecektir. örneğin V3.5 E32'yi desteklemiyorken V4.0 ve V4.7 destekleyebiliyor. kullandığınız modüle uyumlu olup olmadığına bakmanızda fayda olur.

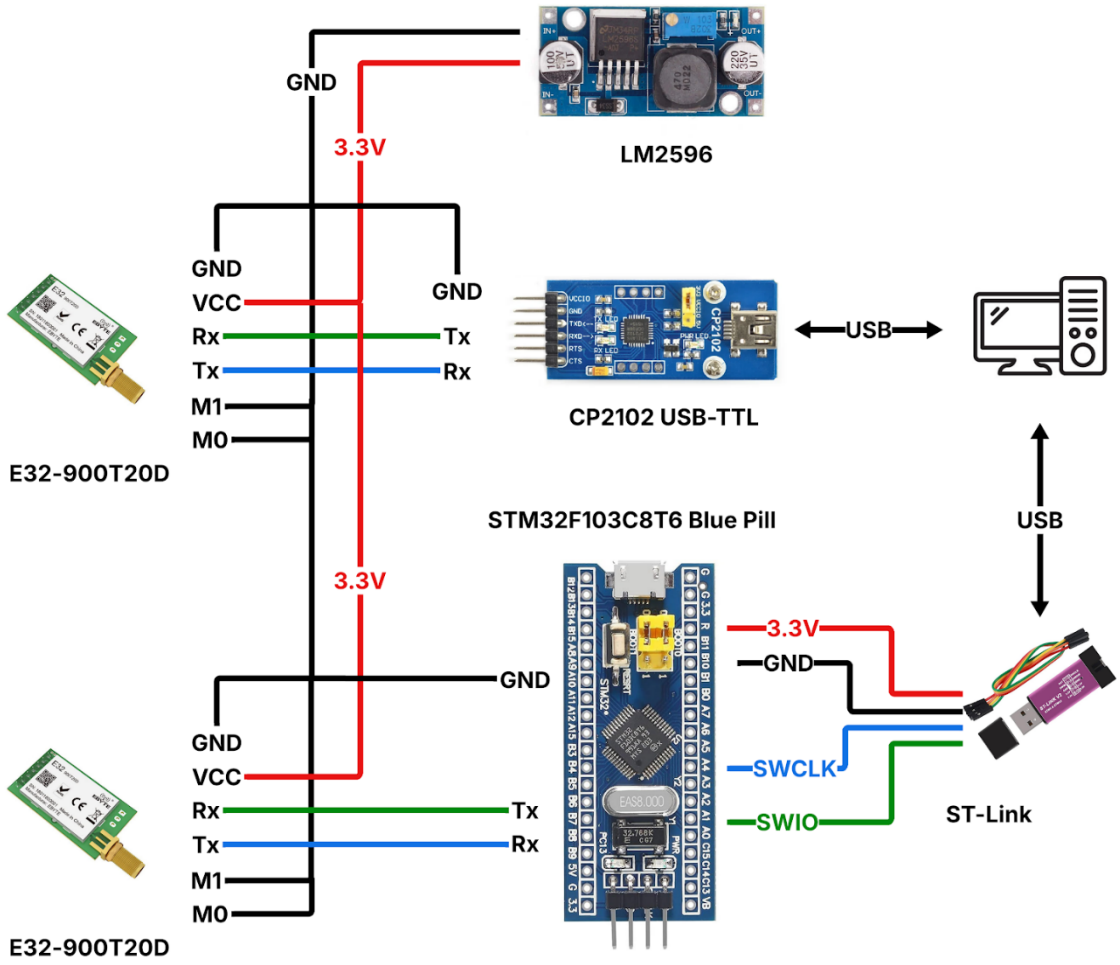
OpenPort'a bastıktan sonra ***GetParam*** butonuna bastıktan sonra modülün içerisinde kayıtlı olan parametre ayarları gözükecektir.

Parametre	Değer	Açıklama
UartRate	9600 bps	Modül ile bilgisayar arasındaki seri haberleşme hızı.
Parity	8N1	8 veri biti, parite yok, 1 dur bitinden oluşur.
AirRate	2400 bps	Havadaki (RF üzerinden) veri iletim hızı. Düşük hız, daha uzun menzil sağlar.
Power	20 dBm	RF çıkış gücü. 20 dBm = 100 mW.
FEC (Error Correction)	Enable	Hata düzeltme mekanizması. Veri güvenliği sağlar, hız düşebilir.
Fixed Mode	Disable	Sabit adresleme kapalı. Yayın veya dinamik adresleme kullanılır.
WOR Timing	250 ms	Uyku modunda sinyal dinleme aralığı.
IO Mode	PushPull	Giriş/çıkış pin modu. Push-pull, daha güçlü çıkış sağlar.
Address	0	Modülün adresi. Diğer modüllerle haberleşmede tanımlayıcıdır.
Channel	6	Kanal numarası. Genelde frekans = 868 MHz + (kanal * 1 MHz) şeklinde hesaplanır.

iki modülün de aynı ayarlara sahip olması gerekmektedir. bu olası sorunları önler. parametreleri değiştirdikten sonra ***SetParam*** dediğimiz takdirde yeni parametre değerleri yüklenir. ilk lorayı konfigüre ettik, ikinci LoRayı da takıp konfigüre ediyoruz.

Artık iki LoRa modülünün haberleşmesi için gereken konfigürasyon yapılmıştır. haberleşmenin gerçekleşebilmesi için de M0 ve M1 pinlerinin GND ile ortaklanması gerekir. bu farklı modüllerde farklı üreticilerde farklılık gösterebilir. Datasheet sayfasını incelemekte fayda olacaktır.

aşağıda bağlantı şeması verilmiştir.



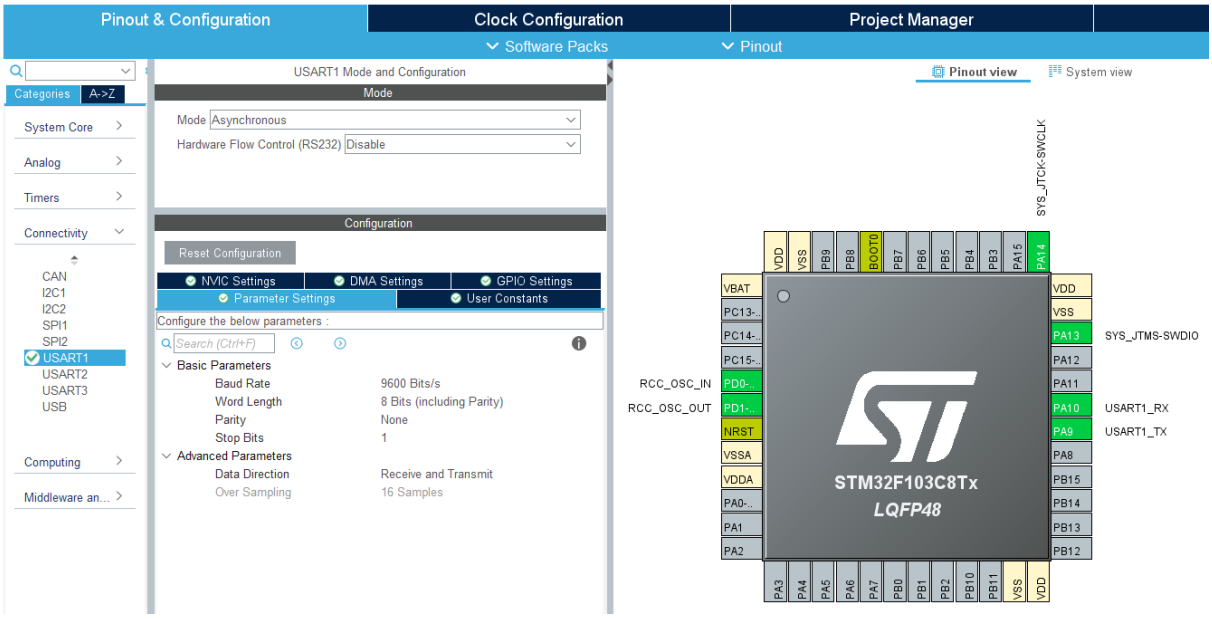
iki türlü kod denemesi gerçekleştireceğiz. bir uygulamada STM32 gönderici ve USB-TTL alıcı olacaktır. diğer uygulamada STM32 Alıcı, USB-TTL gönderici olacaktır.

İlk olarak STM32_Lora_Transmit projesi oluşturup A10 ve A9 pinlerini uart pinleri olacak şekilde konfigüre ediyoruz. STM32 tarafındaki konfigürasyon ayarlarımız LoRa modülü ile uyumlu olmalıdır.

Örneğin;

LoRa modülümüz fabrika ayarı olarak UART 9600 Baud Rate ile haberleşiyor arayüzde değiştirip değiştirmeme durumuna göre en son ki ayarı ne ise STM32 UART konfigürasyonumuz da bununla aynı olması gerekmektedir. Parity Biti LoRa modülünde *None* ise STM32 tarafında da değeri *None* olmalıdır. Lora modülünde *dur biti* 1 ise STM32 tarafında da aynı olması gerekmektedir. aşağıda konfigürasyon ayarlarının ekran görüntüsü bulunmaktadır.

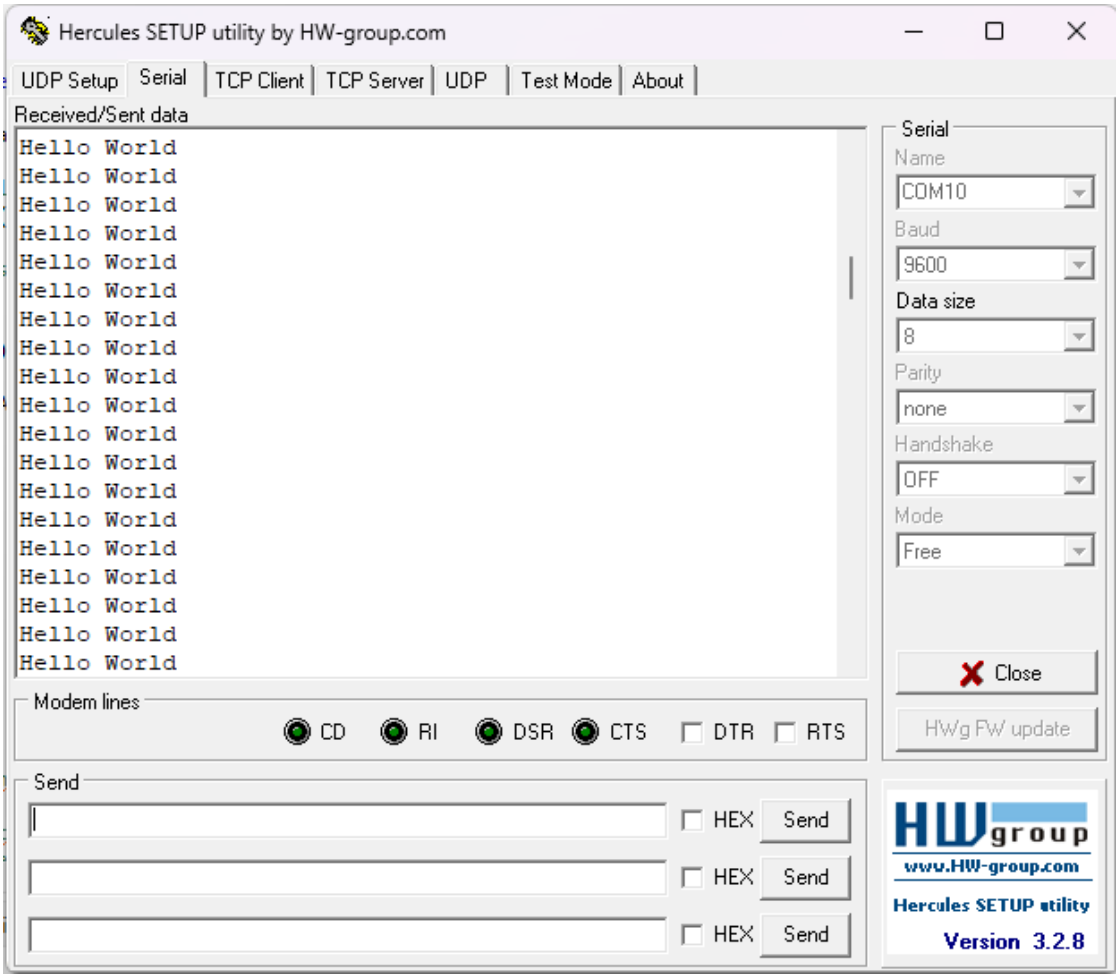
STM32_Lora_Transmit.ioc



main.c

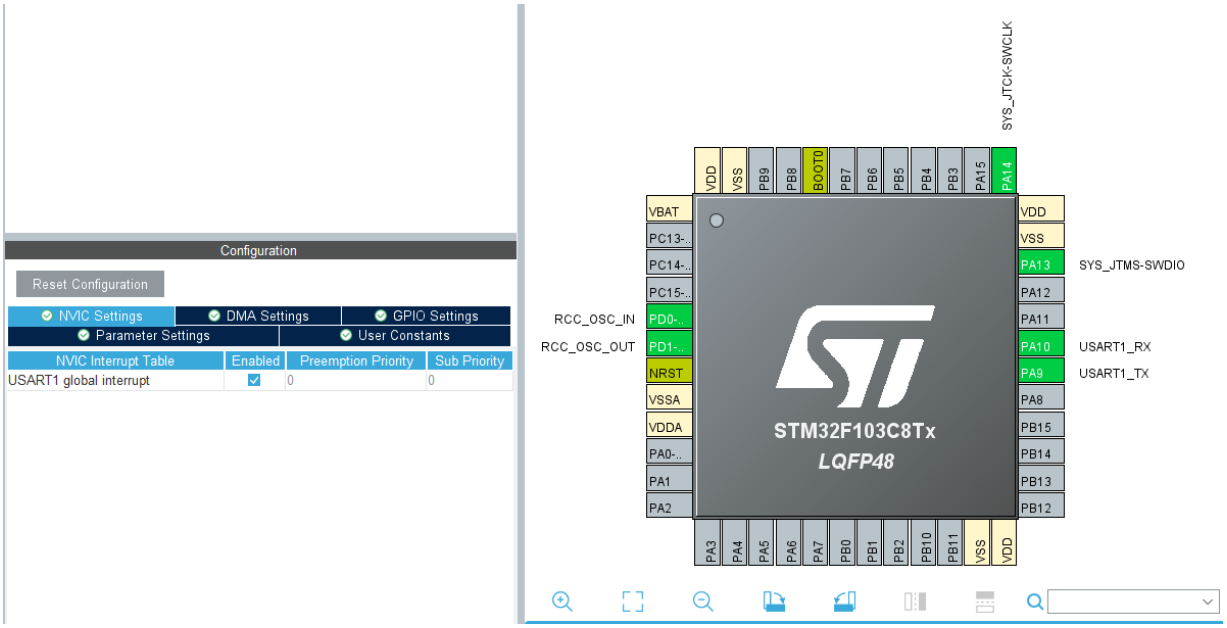
```
/* USER CODE BEGIN 0 */
char byte[] = "Hello World \r \n";
/* USER CODE END 0 */

 /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
        HAL_UART_Transmit(&huart1, (uint8_t*)byte, sizeof(byte), 1000);
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
```

PC çıktısı

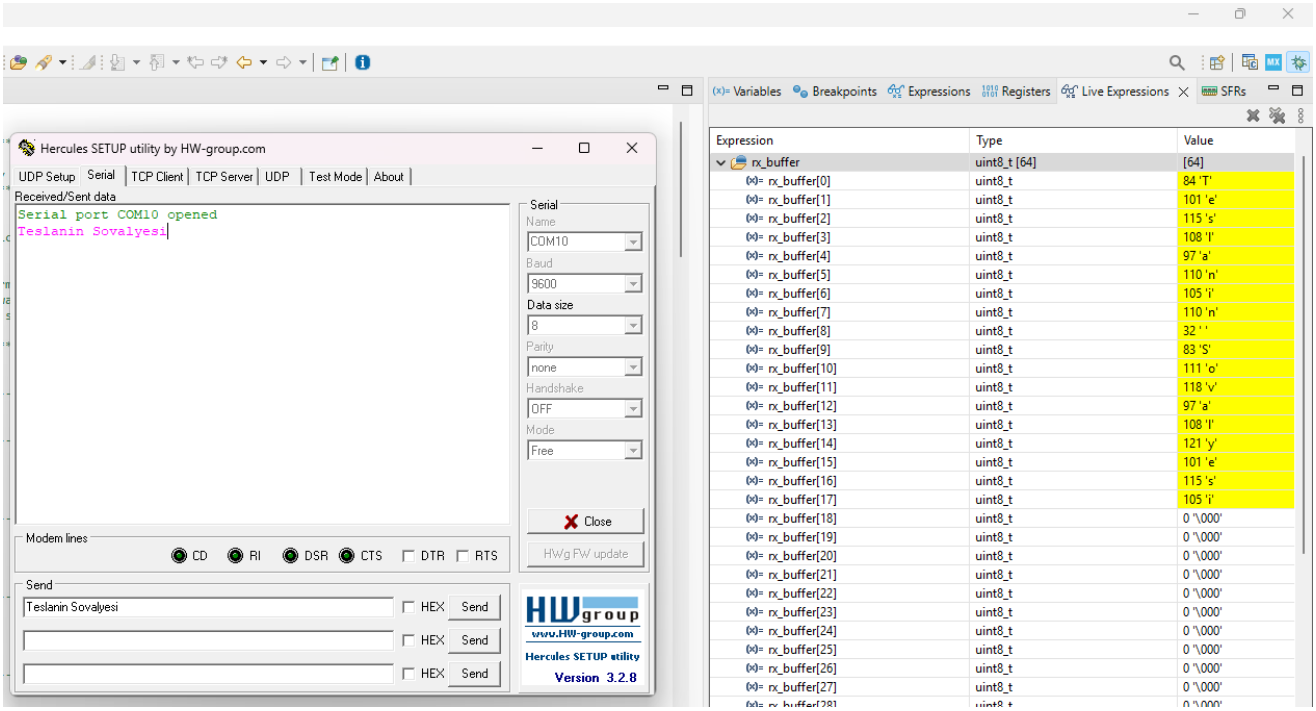


şimdi de USB-TTL üzerinden gönderici ve STM32'nin alıcı olduğu durumu yapılandıralım.

STM32_Lora_Receive.ioc



main.c

```c
/* USER CODE BEGIN 0 */
#define RX_BUFFER_SIZE 64
uint8_t rx_buffer[RX_BUFFER_SIZE];
uint8_t rx_data;
uint8_t rx_index = 0;
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
        if (huart->Instance == USART1)
        {
                if (rx_index < RX_BUFFER_SIZE - 1)
                {
                        rx_buffer[rx_index++] = rx_data;
               if (rx_data == '\n')
               {
                   rx_buffer[rx_index] = '\0';
                   rx_index = 0;
                   //veri burada işlenebilir
               }
          }
                else
                {
               rx_index = 0;
          }
          HAL_UART_Receive_IT(&huart1, &rx_data, 1);
     }
}
/* USER CODE END 0 */
  /* USER CODE BEGIN 2 */
  HAL_UART_Receive_IT(&huart1, &rx_data, 1);
  /* USER CODE END 2 */
```

PC çıktısı